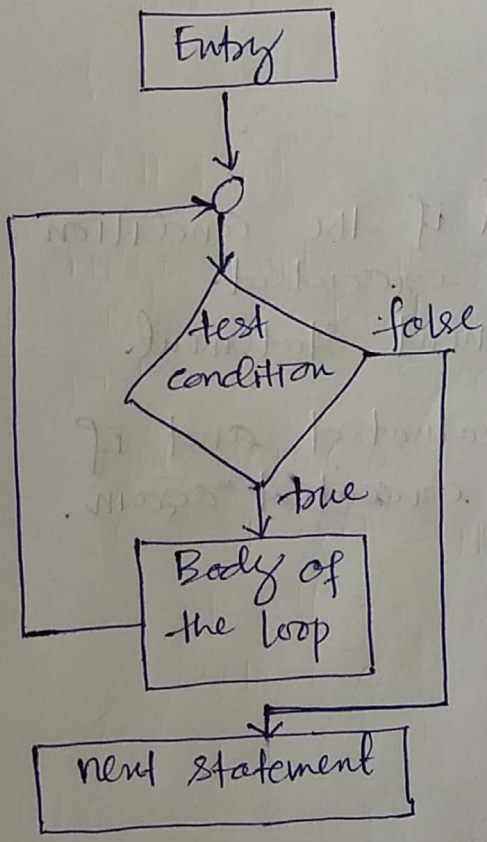
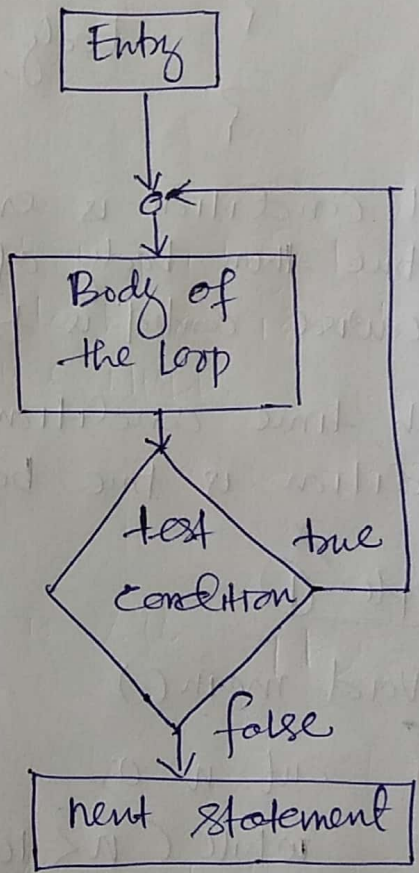
UNIT-2

Loops:

* In looping a sequence of statements are executed until some condition for termination is satisfied.
* A program loop consists of two segments:
  body of the loop and control statement.
* Depending on the position of the control statement the loop can be classified as:
  entry control loop and exit control loop.



(Entry Controlled loop)



(Exit Controlled loop)

Scanned with CamScanner

A loop statement include the following four steps:

1. Setting and initialization of a condition variable.
2. Test the condition of the loop.
3. Execution of statements in the loop.
4. Update the condition variable.

The while loop :

The while loop is an entry-controlled loop statement.
The general form of while loop is :

```
while ( condition)
{
    body of the loop;
}
```

First condition is evaluated and if the condition is true then body of the loop is executed.
Otherwise control is transfered to next statement.

Each time condition will be evaluated and if condition is true body will be executed again.

Example-①

```
void main()
{  int n=0;
   while (n<10)
   {
       printf ("%d \n", n);
       n++;
   }
}
```

Scanned with CamScanner

Example-②: Write a program to find factorial of a number?

```
#include<stdio.h>
int main()
{
    int num, i=1;
    int fact=1;
    printf("Enter any positive integer");
    scanf("%d", &n);
    while(i<=num)
    {
        fact = fact * i;
        i++;
    }
    printf("\n Factorial of %d= %d", num, fact);
    return 0;
}
```

The do-while loop:

In case of do-while loop the body of the loop will execute first. At the end of loop, test condition in while statement is evaluated.

If the condition is true, body will be executed otherwise control will be transfered to next statement after do-while loop.

The general form of do-while loop is:

```
do
{
    body of the loop;
} while(test-condition);
```

Scanned with CamScanner

Example: (1)

```
void main()
{
    int a = 10;
    do
    {
        printf("Value of a = %d", a);
        a++;
    } while (a<20);
}
```

Example (2) for multiplication table.

```
#include<stdio.h>
int main()
{   int row, col, y;
    row = 1;
    printf("Multiplication table\n");
    do
    {
        col = 1;
        do
        {
            y = row * col;
            printf("%d", y);
            col = col+1;
        } while (col <= 10);
        printf("\n");
        row = row + 1;
    } while (row <= 10);
}
```

Scanned with CamScanner

The for loop:

The general form of for loop is:

```
for (initialization; condition; update)
{   body of the loop;
}
```

The execution of the for loop statement is as follows:

1. Initialization of control variable is done first.
2. Then the test condition is evaluated. If the condition is true, the body of the loop is evaluated; otherwise loop is terminated.
3. After body of the loop is executed, the control is transfered to updation of the control variable. Then again condition is tested.
4. If the condition is satisfied, the body of the loop is again executed, else control is transfered to the next statement after the loop.

Example:

```
#include <stdio.h>
int main()
{   int sum=0, x;
    for (x=1; x<=50; x++)
    {   sum = sum + x;
    }
    printf ("1+2+3+...+50 = %d", sum);
    return 0;
}
```

Scanned with CamScanner

Example - ②

```
#include <stdio.h>
int main()
{
    int i, j;
    for(i=1; i<5; i++)
    {
        for(j=1; j<=i; j++)
            printf("*");
        printf("\n");
    }
    return 0;
}
```

Output

```
*
* *
* * *
* * * *
* * * * *
```

Q. Try the following pattern ?

(1)

```
    *
   * *
  * * *
 * * * *
* * * * *
```

(2)

```
    *
   * * *
  * * * * *
 * * * * * * *
* * * * * * * * *
```

(3)

```
    1
   2 2 2
  3 3 3 3 3
 4 4 4 4 4 4 4
5 5 5 5 5 5 5 5 5
```

(4)

```
    1
   2 1 2
  3 2 1 2 3
 4 3 2 1 2 3 4
5 4 3 2 1 2 3 4 5
```

Scanned with CamScanner

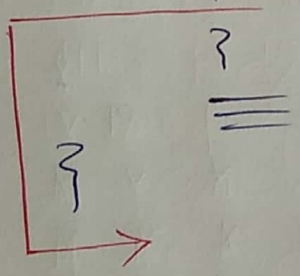
The break and continue :

The break statement terminates the loop immediately when it is encountered.

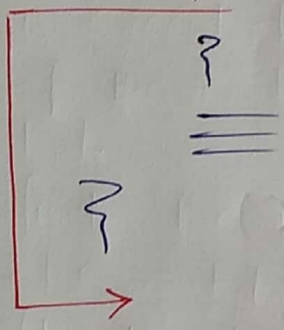
The break is used with decision making statement such as if-else.

The general form of break are as follows :

```
while (condition)
{
    ≡≡≡
    if (condition)
    {
        break;
    }
    ≡≡≡
}
```



```
for (init; cond; update)
{
    ≡≡≡
    if (condition)
    {
        break;
    }
    ≡≡≡
}
```



Example :

```
void main()
{  double num, Sum = 0.0;
   int i;
   for (i=1; i<=10; i++)
   {  printf ("Enter a number");
      scanf ("%lf", &num);
      if (num < 0.0)
      {
         break;
      }
      Sum = Sum + num;
```

Scanned with CamScanner

```
    printf("Sum = %lf", Sum);
}
```

The continue statement skips some statements inside the loop.
The continue statement is used with decision making statement such as if-else.
The general form of continue statement is :

```
while (condition)
{
    ==
    if (condition)
    {
        continue;
    }
    ==
}
```

```
for(init; condn; update)
{
    ==
    if (condition)
    {
        continue;
    }
    ==
}
```

Example :

```
void main()
{
    int i;
    double num, Sum = 0.0;
    for (i=1; i<=10; i++)
    {
        printf("Enter a number\n");
        Scanf("%lf", &num);
        if (num < 0.0)
        {
            continue;
        }
        Sum = Sum + num;
    }
    printf("\n Sum = %lf", Sum);
}
```

Scanned with CamScanner

Arrays

Array is a collection of variables of the same type referred through a ~~Collection~~ common name.

In c all elements of an array are stored in contiguous memory location.

A specific element in an array is accessed by an index.

Array can have one or multiple dimensions.

i.e One dimensional arrays
Two dimensional arrays
Multi dimensional arrays

One-Dimensional arrays:

In one dimensional array a list of items can be given one variable name.

Like other variables array must be declared before use, so that compiler can allocate sufficient space for them in the memory.

The general form of one-dimensional array is:

```
datatype variable_name [size];
```

Here datatype specifies the type of element that will be contained in the array. i.e int, float etc

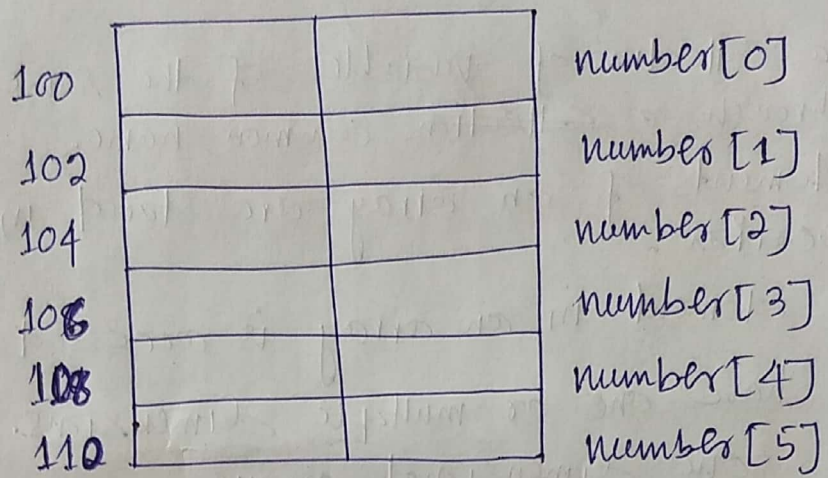
Size of array defines maximum how many elements the array can hold.

Example:

```
double balance [100];
float height [50];
char name [20];
```

Scanned with CamScanner

Consider int number[6];

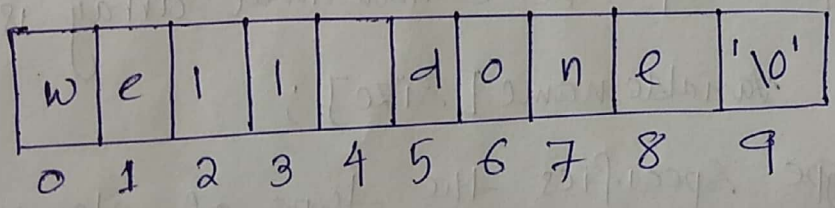


The most common use of one-dimensional array is character string.

In C, a string is a null terminated character array. When declaring a character array, you need to declare one character longer to store null character.

Example:

```
char name[10];
name = "well done";
```



When the compiler sees a character string, it terminates the string with an additional null character.

Scanned with CamScanner

Initialization of one-dimensional array:

After an array is declared, its elements must be initialized, otherwise it will contain garbage value.

An array can be initialized either in compile time or in run-time.

The general form of compile-time initialization is:

datatype array_name[size] = {list of values};

- The values are separated by comma.

Example:

```
int number[3] = {0, 0, 0};
```

If the number of values in the list is less than the size of the array then the remaining elements will be set to zero.

Example:

```
float total[5] = {0.0, 15.5};
```

If you ommit the size of array the compiler allocates enough space for all initialized elements.

```
int counter[] = {1, 1, 1, 1};
Char name[] = "John";
```

An array can be explicitly initialized at run-time. This approach is usualy used to initialize a large array.

```
int num[100];
for (i=0; i<100; i++)
{   num[i] = 2;
}
```

We can also use Scanf() function to initialize an array.

```
int num[10];
for (i=0; i<10; i++
        Scanf("%d", &num[i]);
```

Scanned with CamScanner

Example :

```
void main()
{
    int num[10];
    int sum = 0, count, i;
    float avg = 0.0;   count = 10;
    printf("Enter 10 numbers\n");
    for(i=0; i< count; i++)
    {
        printf("num[%d] = ", i);
        scanf("%d", &num[i]);
        sum += num[i];
    }
    avg = sum/count;
    printf("\n Average of 10 numbers entered
            is : %f", avg);
}
```

Example

```
void main()
{   int x[100];
    int i, sum=0;
    for(i=0; i<10; i++)
    {   x[i] = i;
    }
    for(i=0; i<10; i++)
    {   sum = sum + x[i] * x[i];
    }
    printf("\n Sum of squares of 1st 10
            num is %d", sum);
}
```

Scanned with CamScanner

Two Dimensional Arrays :

A two dimensional array is essentialy, an array of one dimensional arrays.

Two-D array can be declared as follows :

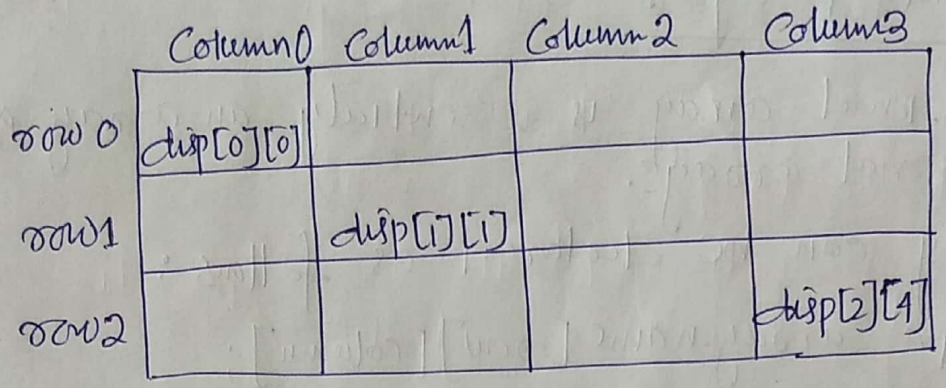
datatype array-name [row][column];

2-D array in C programming is also known as matrix.

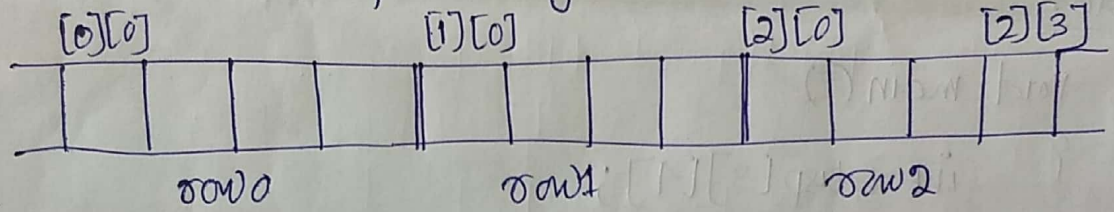
Example :

```
void main()
{
    int disp[3][4];
    int i,j;
    for(i=0; i<3; i++)
    {
        for(j=0; j<4; j++)
        {
            printf("Enter value to display:
                    disp[%d][%d]:", i,j);
            scanf("%d", &disp[i][j]);
        }
    }
    for(i=0; i<3; i++)
    {
        for(j=0; j<4; j++)
        {
            printf("%d", disp[i][j]);
        }
        printf("\n");
    }
```

Scanned with CamScanner



The elements of an array are stored contiguously in increasing order of memory location in a single unit.



Initialization of 2D arrays:

Like 1-D array 2-D array can be initialized by using a list of initial values enclosed in braces during declaration.

```
int matrix[2][3] = { 0,0,0, 1,1,1 };
int table[2][3] = { {0,0,0}, {1,1,1} };
```

When the array is completely initialized with all values, we need not specify the size of first dimension.

```
int table[][] = { {0,0,0},
                  {1,1,1}
                };
```

If the values are missing in an initializer list, they are automatically set to zero.

```
int table[2][3] = { {1,1},
                    {2} };
```

Scanned with CamScanner

Example: Sum of two matrix of order 2×2:

```
void main()
{   int a[2][2], b[2][2], c[2][2];
    printf("Enter the elements of 1st matrix\n");
    for(i=0; i<2; ++i)
        for(j=0; j<2; j++)
        { printf("Enter a[%d][%d]:", i+1, j+1);
          scanf("%d", &a[i][j]);
        }
    printf("Enter the elements of 2nd matrix\n");
    for(i=0; i<2; i++)
      for(j=0; j<2; j++)
        { printf("Enter b[%d][%d]", i+1, j+1);
          scanf("%d", &b[i][j]);
        }
    for(i=0; i<2; ++i)
      for(j=0; j<2; ++j)
        {   c[i][j] = a[i][j] + b[i][j];
        }
    for(i=0; i<2; ++i)
      for(j=0; j<2; ++j)
        { printf("%d", c[i][j]);
        }
        printf("\n");
}
```

Scanned with CamScanner

Multidimensional Arrays:

C allows arrays of more than two dimensions. The general form of multidimensional array declaration is:

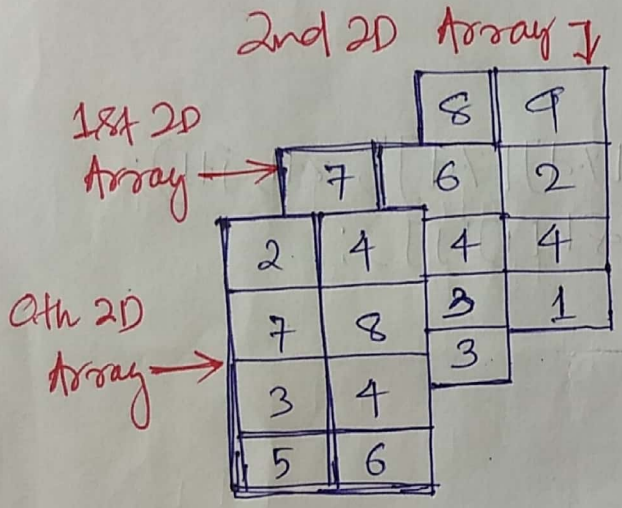
datatype array_name [size1] [size2] ... [sizeN];

Example: int table[5][4][6][2];

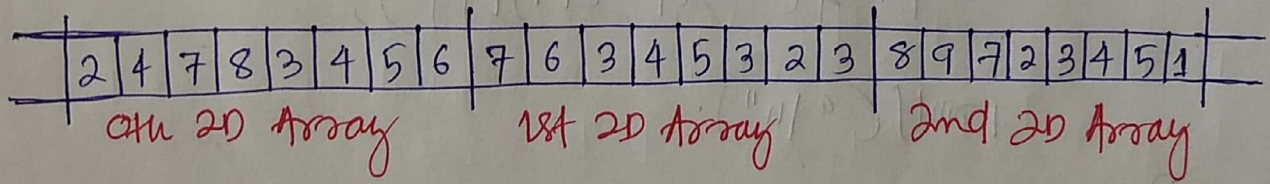
In practice multidimensional array is rarely used.

Example:

```
int arr[3][4][2] = { { { 2,4 },
                       { 7,8 },
                       { 3,4 },
                       { 5,6 } },
                     { { 7,6 },
                       { 3,4 },
                       { 5,3 },
                       { 2,3 } },
                     { { 8,9 },
                       { 7,2 },
                       { 3,4 },
                       { 5,1 } } };
```



In memory the multidimensional array is stored as:



Scanned with CamScanner

Character Arrays and Strings:

Character strings are used to build meaningful and readable programs.

The common operations performed on character string are:

- Reading and writing strings
- Combining strings together.
- Copying one string to another
- Comparing strings for equality
- Extracting a portion of string.

Declaring and Initializing string variables:

The general form of declaring string variable is:

```
char string_name[size];
char name[20];
```

When the compiler assigns a character string to a character array, it automatically supplied a null character ('\0') at the end of the string.

```
char city[10] = "NEW DELHI";
char str1[] = "Thank you";
char str2[3];
str2 = "Good"; // error cannot separate declaration from initialization.
char s1[4] = "abc";
char s2[4];
s2 = s1; // error cannot assign a string to another.
```

Scanned with CamScanner

Reading strings from terminal :

Reading a single character from terminal can be done using getchar() function.

Syntax: variable_name = getchar();

```
char name;
name = getchar();
```

Example:

```
void main()
{
    char line[20], ch;
    int c=0;
    printf("Enter text. Press ENTER at the end\n");
    do {
        ch = getchar();
        line[c] = ch;
        c++;
    } while (ch != '\n');
    c = c-1
    line[c] = '\0';
    printf("\n %s \n", line);
}
```

Here, the reading is terminated when the new line character (\n) is entered and the null character is then inserted at the end of the string.

Scanned with CamScanner

Another method of reading a string containing white space is gets() function.

Syntax:

```
gets (string);
```

Example:

```
char line[20];
gets (line);
printf ("%s", line);
```

Be careful not to input more character than that can be stored in the string variable used because c doesnot check any array bounds. So it may cause problem.

C doesn't provide operator that work on strings directly. we cannot assign one string to another directly.

i.e we cannot write:

```
char sto[10];
sto = "ABC";
sto = sto2;
```

Example: Program to copy one string from another.

```
#include<stdio.h>
int main()
{
    char sto1[50], sto2[50];
    int i;
    printf ("Enter a string\n");
    gets(sto2);
    for( i=0; sto2[i]!='\0'; i++)
        sto1[i] = sto2[i];
```

Scanned with CamScanner

```
    str1[i] = '\0';
    printf("\n %s \n", str1);
    printf("Number of character = %d \n", i);
  return 0;
}
```

Writing strings to screen :

In c putchar function is used to print the value of character strings.

Example :

```
char ch = 'A';
putchar(ch);
```

```
char name[5] = "GOOD";
for(i=0; i<5; i++)
    putchar(name[i]);
putchar('\n');
```

Another function puts() is used to print the entire string at a time.

Example :

```
char line[80];
gets(line);
puts(line);
```

Concatenation and Comparision of Strings :

we cannot directly join two strings using addition operator :

```
str3 = str1 + str2; // error
str2 = str1 + "hello"; //error
```

These above statements are not valid.

Scanned with CamScanner

Example: Program to copy one string from another:

```
#include<stdio.h>
int main()
{   int i,j,k;
    char first[10] = "FAKIR";
    char sec[10] = "MOHAN";
    char last[10] = "COLLEGE";
    char name[30];
  for(i=0; first[i]!='\0'; i++)
        name[i] = first[i];
  name[i] = ' ';
  for(j=0; sec[j]!='\0'; j++)
        name[i+j+1] = sec[j];
  name[i+j+1] = ' ';
  for(k=0; last[k]!='\0'; k++)
        name[i+j+k+2] = last[k];
  name[i+j+k+2] = '\0';
  puts(name);
  return 0;
}
```

C doesnot permit the comparision of two strings directly

i.e if (name1 == name2)

or if (name1 == "ABC")

are not permited.

So it is necessary to compare two strings character by character.

Scanned with CamScanner

Example:

```
char str1[] = "ABC";
char str2[] = "XYZ";
int i=0;
while (str1[i] == str2[i] &&
       str1[i] != '\0' && str2[i] != '\0')
    i=i+1;

if (str1[i] == '\0' && str2[i] == '\0')
    printf ("Strings are equal");
else
    printf ("Strings are not equal");
```

Pointers

Introduction :

Pointers are variables; contain memory addresses as their values. The memory addresses are the locations in the computer memory where program instructions and data are stored.

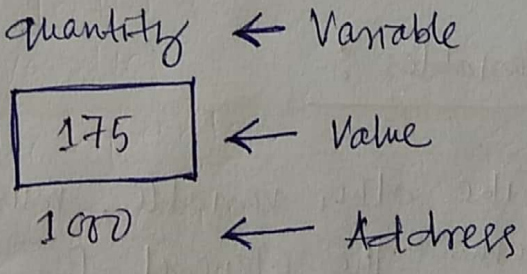
Pointers are used to access and manipulate data stored in the memory.

The computers memory is a sequential collection of storage cells and each storage cell is known as byte, each byte has a number called "address" associated with it.

Consider the following statement :

```
int quantity = 175;
```

- Whenever we declare a variable, the system allocates appropriate amount of memory to hold the value of variable.
- Let us assume that the system has choosen the address 1000 for quantity.



Since memory addresses are simply numbers they can be assigned to some variables that can be stored in memory.

So the variables that hold memory addresses are called pointer variables.

Scanned with CamScanner

Accessing the address of a variable:

We use "&" operator to access the address of the variable associated with it. So "&" operator is called "address of" operator.

The & operator can be used only with a simple variable or an array element.

If x is an array, the expressions such as:

&x[0] and &x[i+3]

are valid and represent the address of 0th and (i+3)th element of x.

Example: Print the address of variable with its value.

```
void main()
{
    char a = 'A';
    int x = 100;
    float p = 10.25;
    printf("%c is stored at address %u.\n", a, &a);
    printf("%d is stored at address %u.\n", x, &x);
    printf("%f is stored at address %u.\n", p, &p);
}
```

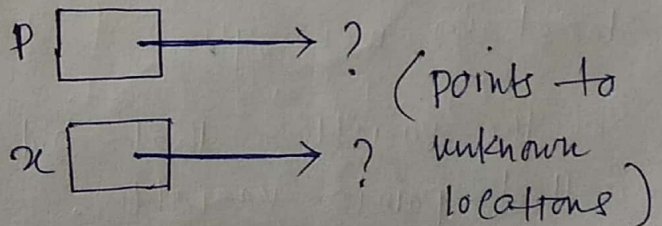
Declaring pointer variables:

Pointer variables, like other variable, must be declared before we use them. The general format is:

```
data_type *pointer_name;
```

Example:

```
int *p;
float *x;
```



Scanned with CamScanner

Pointers can be declared in different style.

int * p; or int* p; or int *p;

Initialization of pointer variables:

Once a pointer variable has been declared we can use the assignment operator to initialize the variable:

Example: int quantity;
int *p;
p = &quantity;

- we can also combine the initialization with the declaration:

int *p = &quantity;

It is also possible to combine the declaration of variable and pointer variable with initialization of the pointer.

int x, *p = &x;

We could also define a pointer variable with an initial value NULL or 0.

int *p = NULL;
int *q = 0;

Accessing a variable through its pointer:

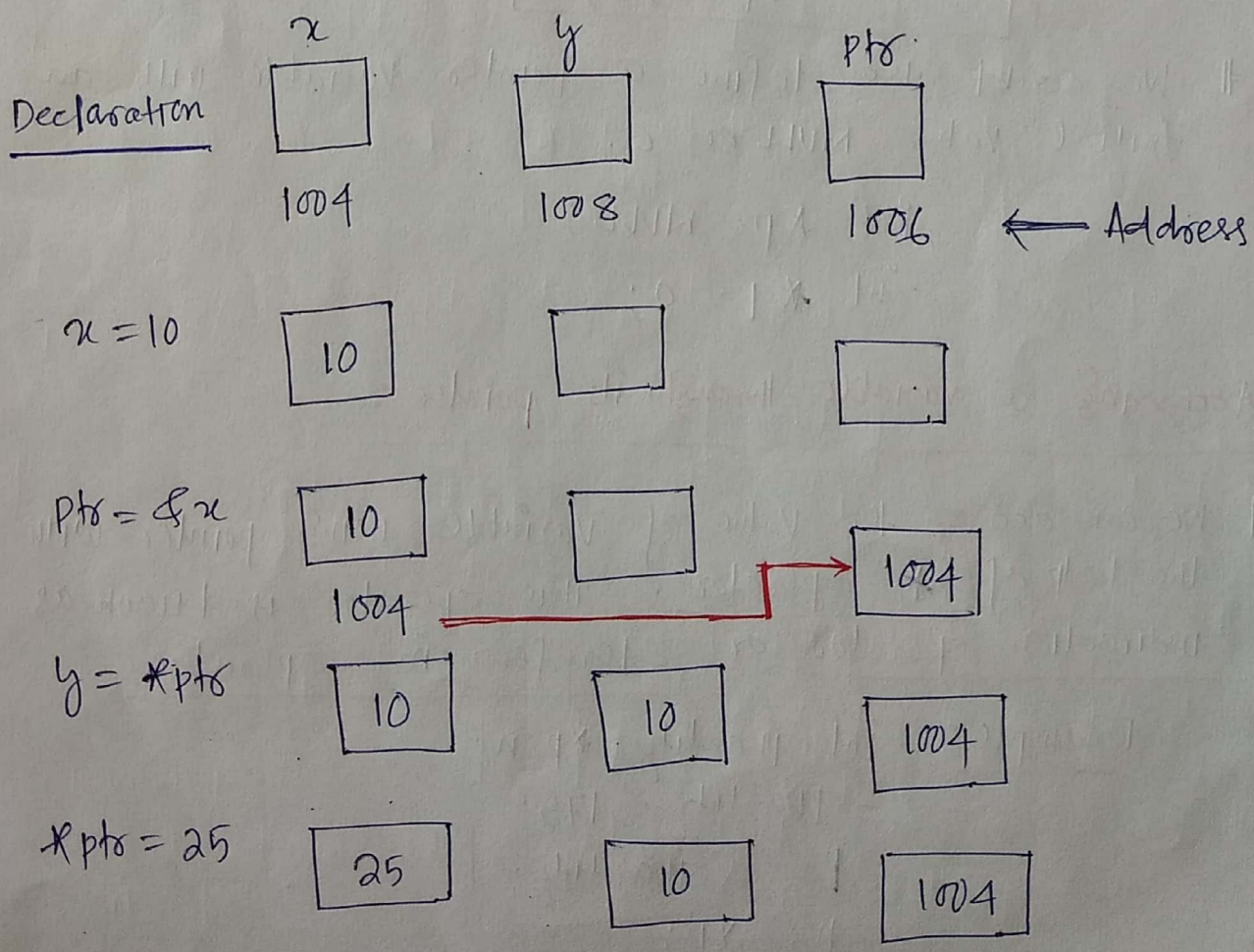
We can access the value of variable using pointer with the help of "*" operator. This operator is known as indirection operator or dereferencing operator.

Example: int quantity, *p, n;
quantity = 175;
p = &quantity;
n = *p;

Scanned with CamScanner

Example:

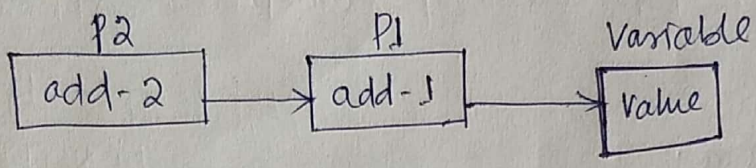
```
void main()
{
    int x, y;
    int *ptr;
    x = 10;
    ptr = &x;
    y = *ptr;
    printf("Value of x is %d\n\n", x);      // 10
    printf("%d is stored at address %u\n", x, &x);
    printf("%d is stored at address %u\n", *&x, &x);
    printf("%d is stored at address %u\n", ptr, &ptr);
    printf("%d is stored at address %u\n", y, &y);
    *ptr = 25;
    printf("\n\nNow x = %d\n", x);          // 25
}
```



Scanned with CamScanner

Chain of pointers :

It is possible to make a pointer to point another pointer, thus creating a chain of pointers.



A variable that is a pointer to a pointer must be declared using additional indirection operator symbols infront of the name :

```
int **p2;
```

— Here p2 is not pointer to integer, but to a pointer to an integer pointer.

Example :

```
void main()
{
    int x, *p1, **p2;
    x = 100;
    p1 = &x;
    p2 = &p2;
    printf("%d", **p2); // 100
}
```

Pointer expressions :

Like other variables, pointers can be used in expressions.

C allows us to add integers to or subtract integers from pointers. we can also subtract one pointer from another.

If p1 and p2 are both pointers to the same array then p2-p1 gives the number of elements between p1 & p2.

Pointers can be compared using the relational operators. Expressions such as (p1 > p2), (p1 == p2) and (p1 != p2) are allowed where p1 and p2 are two pointers.

Scanned with CamScanner

Example :

```
void main()
{
    int a, b, *p1, *p2, x, y, z;
    a = 12;
    b = 4;
    p1 = &a;
    p2 = &b;
   x = *p1 * *p2 - 6;
   y = 4 * -*p2 / *p1 + 10;
   // ((4 * (-(*p2))) / (*p1)) + 10
   printf("a = %d, b = %d\n", a, b);  // 12  4
   printf("x = %d, b = %d\n", x, y);  // 42  9
   *p2 = *p2 + 3;
   *p1 = *p2 - 5;
   z = *p1 * *p2 - 6;
   printf("\n a = %d, b = %d", a, b);  // 2  7
   printf("z = %d\n", z);  // 8
}
```

Pointer increment and scale factor :

Expression like p1++; will cause the pointer p1 to point to the next value of its type.

Example : p1 is an integer pointer with value 2000.

p1 = p1+1; will be 2002 not 2001 (int 2 bytes)

If p2 is a float type pointer with address 1000.

p2 = p2+1; will be 1004 not 1001.

So when we increment a pointer its value is increased by the length of datatype that it points to. This length is called the "scale factor".

Scanned with CamScanner

Rules of pointer Operations :

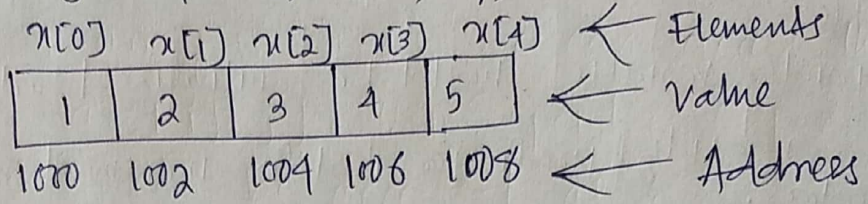
(1) A pointer variable can be assigned the address of another variable.
(2) A pointer variable can be assigned the value of another pointer variable.
(3) A pointer variable should be initialized with NULL or 0.
(4) We can apply prefix or post fix increment and decrement operators.
(5) An integer value may be added or subtracted from a pointer variable.
(6) When two pointers point to the same array, one pointer variable can be subtracted from another.
(7) When two pointers point to the objects of same type, they can be compared with relational operators.
(8) A pointer cannot be multiplied by a constant.
(9) Two pointer variables cannot be added.
(10) A value cannot be assigned to an arbitrary address. (i.e &x = 10; is illegal).

Scanned with CamScanner

Pointer and Arrays:

Suppose we declare an array x as follows:

```
int x[5] = { 1, 2, 3, 4, 5 };
```



The array name x is defined as a constant pointer pointing to the first element x[0].

$x = \&x[0] = 1000$

we can access the array elements using a pointer:

```
int *p;
p = &x[0];
p+1 = &x[1]
  :
p+4 = &x[4]
```

The address of an element is calculated using its index and the scale factor of the data type.

address of x[3] = base address + (3 × scale factor of int)
= 1000 + (3×2) (index)
= 1006

Example:

```
void main()
{
    int *p, sum, i=0;
    int x[5] = { 5, 9, 6, 3, 7 };
    p = x;
    printf("Element Value Address \n");
```

Scanned with CamScanner

```
while (i<5)
{
    printf ("x[%d] %d %d", i, *p, p);
    sum = sum + *p;
    i++; p++;
}
printf ("\n Sum = %d\n &x[0] = %u\n", sum, &x[0]);
printf ("\n p= %u", p);
}
```

Pointer representation of array:

We can declare a pointer that can point to whole array instead of only one element of the array.

Syntax: `datatype (*array_name)[size];`

Example: `int (*ptr)[10];` (ptr: pointer_name)

Here ptr is a pointer that can point to an array of 10 integers. The type of ptr is "pointer to an array of 10 integers".

Example:

```
int main()
{
    int *p;
    int (*ptr)[5];
    int arr[5] = { 1,2,3,4,5 };
    p = arr;
    ptr = &arr;
    printf ("p= %u, ptr= %u", p, ptr);
    p++; ptr++;
}   printf ("p= %u, ptr= %u", p, ptr);
```

Scanned with CamScanner

pointers and 2-D arrays:

The elements of 2-D array can be accessed with the help of pointer representation.

Scoppose the 2D array is "arr", we can access the elements as arr[i][j]. Using pointer expression the same array can be accessed as :

((arr+i)+j)

Consider one example:

```
int arr[3][4] = { {1,2,3,4},{5,6,7,8},
                  {9,10,11,12} };
```

arr

	col 0	col 1	col 2	col 3
row 0	1	2	3	4
row 1	5	6	7	8
row 2	9	10	11	12

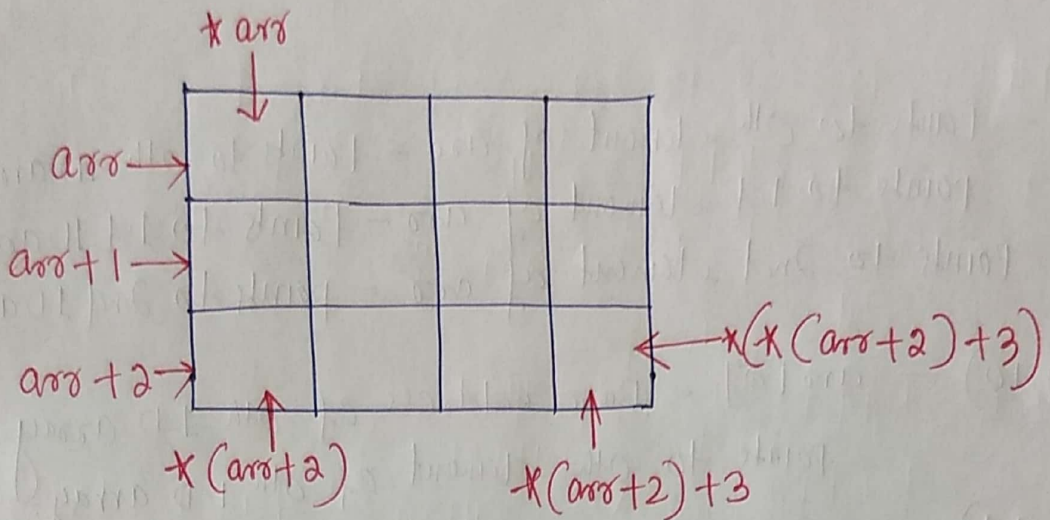
"arr" is a constant pointer

we know that the name of an array is a constant pointer that points to 0th 1-D array.

Here arr is a "pointer to an array of 4 integers", and arr+1 will point to next one-D array and arr+2 will be the 3rd 1-D array.

arr →	1	2	3	4
arr+1 →	5	6	7	8
arr+2 →	9	10	11	12

Scanned with CamScanner



Example :

```
#include<stdio.h>
int main()
{
    int arr[3][4] = { {1, 2, 3, 4},
                      {5, 6, 7, 8},
                      {9, 10, 11, 12} };
    int i, j;
    for(i=0; i<3; i++)
    {
        printf("Address of %dth array = %u %u",
                i, arr[i], *(arr+i));

        for(j=0; j<4; j++)
            printf("%d %d", arr[i][j],
                    *(*(arr+i)+j));
    }
    return 0;
}
```

Scanned with CamScanner

arr - Points to 0th element of arr - Points to 0th 1-D array
arr+1 - points to 1st element of arr - Points to 1st 1D array
arr+2 - Points to 2nd element of arr - points to 2nd 1-D array

*(arr+0) - arr[0] - Base address of 0th 1-D array.
points to 0th element of 0th 1-D array.

*(arr+1) - arr[1] - Base address of 1st 1D array
points to 0th element of 1st-1-D array.

*(arr+2) - arr[2] - Base address of 2nd 1-D array
points to 0th element of 2nd-1-D array.

To access an individual element of over 2D array we should able to access jth element of ith 1-D array.

Since the base type of *(arr+i) is int and it contains the address of 0th element of ith 1-D array we can get the address of subsequent elements in the ith 1-D array by adding integer value to *(arr+i).

So, *(arr+i)+j will represent the address of jth element of ith 1-D array.

arr : Points to 0th 1-D array
*arr : points to 0th element of 0th 1-D array
(arr+i) : Points to ith 1-D array
*(arr+i) : Points to 0th element of ith 1-D array
*(arr+i)+j : points to jth element of ith 1-D array
((arr+i)+j) : Reproesents the value of jth element of ith 1-D array.

Scanned with CamScanner

Array of pointers :

Like array of basic data types, we can declare array of pointers types also.

Consider the example :

```
#include <stdio.h>
int main()
{
    int arr[] = { 5, 10, 20, 40, 80};
    int i;
    int *ptr[5];
    for( i=0; i<5; i++)
    {
        ptr[i] = &arr[i];
    }
    for (i=0; i<5; i++)
    {
        printf ("Array[%d] = %d\n", i, *ptr[i]);
    }
    return 0;
}
```

output

```
Array[0] = 5
Array[1] = 10
Array[2] = 20
Array[3] = 40
Array[4] = 80
```

Scanned with CamScanner

Limitations of array :

1. Array is a static data structure
2. Memory allocated during compile time.
3. Once memory is allocated it cannot be changed at run.
4. Elements belongs to different data types cannot be stored in array.
5. Inserting element and deletion of element from an existing array is difficult process.
6. Process of checking the limit of array is called Bound checking and c doesnot perform Bound checking for arrays.
7. Memory will be wasted is a large array is declared for a few data values.

Scanned with CamScanner

Dangling, Generic, Null and Wild Pointers :

1. Dangling Pointer :

A pointer pointing to a memory location that has been deleted or freed is called dangling pointer.

Example : // De-allocation of memory

```
#include <stdlib.h>
#include <stdio.h>
int main()
{  int *ptr = (int *) malloc (sizeof (int));
   free (ptr);    // After free, ptr becomes a dangling pointer
   ptr = NULL;  // No more dangling pointer.
}
```

Example : // Function Call

```
#include <stdio.h>
int* fun()
{   int x = 5; // make it static int x = 5;
    return &x;
}
int main()
{    int *p = fun();
     printf ("%d", *p);
     return 0;
}
```

Scanned with CamScanner

Example: Variable goes out of scope.

```
void main()
{ int *ptr;
  =
  { int ch;
    ptr = &ch;
  }
// Here ptr is dangling pointer.
```

2. Generic Pointer :

Generic pointer is a specific pointer type called void pointer. Generic pointer points to some data location in memory which does not have any specific type.

If we assign address of char data type to void pointer it will become char pointer, if int data type then int pointer and so on.

Any pointer type can be convertible to a void pointer so it can point to any value.

Note:

1. Void pointers cannot be dereferenced. It can however be done using type casting the void pointer.
2. Pointer arithmetic is not possible on pointers of void due to lack of concrete value and size.

Scanned with CamScanner

Example :

```
int main()
{
    int x = 4;
    float y = 5.5;
    void *ptr;
    ptr = &x;
  printf("Integer variable is = %d", *((int*) ptr));
    ptr = &y;
  printf("\n float variable is = %f", *((float*)ptr));
  return 0;
}
```

3. Null Pointer :

NULL pointer is a pointer which is pointing to nothing. When we donot have address to be assigned to a pointer then we can simply use NULL.

```
int main()
{
    int *ptr = NULL;
    printf("The value of ptr is %u", ptr);
    return 0;
}
```

Note

1. NULL vs Uninitialized pointer : An uninitialized pointer stored undefined value. A null pointer stores a defined value that is not a valid address of any member.
2. NULL vs Void pointer : NULL pointer is a value but void pointer is a type.

Scanned with CamScanner

4. Wild Pointer :

A pointer which has not been initialized to anything (not even NULL) is known as wild pointer. The pointer may be initialized to a value that may not be a valid address.

```
int main()
{   int *p;  // wild pointer
    int x=10;
     p= &x;
    return 0;
}
```

